École Supérieure Privée d'Ingénierie et de Technologies

Module AWD – MT-41 · Applications Web Distribuées

Rapport de conception

Mini-projet microservices – Validation 1

MedFinder

Votre solution intelligente pour trouver vos médicaments

Réalisé par :
Jihed Khlifi
Abdelhamid Ayadi
Ferdaouas Oueslati
Insaf Makhloufi

Groupe : Gen Z
Classe : 4DASI11
Encadré par : Mme Ons Ben Salah

10 novembre 2026 · Année universitaire 2026 – 2027

Table des matières

Introduction générale **5**

1 Présentation du projet **6**

1.1 Contexte . . . 6
1.2 Problématique . . . 6
1.3 Solution proposée . . . 6
1.4 Objectifs . . . 7
1.5 Acteurs et besoins fonctionnels . . . 7
1.6 Besoins non fonctionnels . . . 8
1.7 Équipe . . . 8

2 Conception fonctionnelle **9**

2.1 Diagramme global de cas d'utilisation . . . 9
2.2 Diagramme de classes global . . . 10
2.3 MS1 · Pharmacies . . . 10
2.3.1 Cas d'utilisation . . . 11
2.3.2 Diagramme de classes . . . 11
2.3.3 Diagramme de séquence : créer puis valider une pharmacie . . . 12
2.4 MS2 · Médicaments . . . 12
2.4.1 Cas d'utilisation . . . 13
2.4.2 Diagramme de classes . . . 14
2.4.3 Diagramme de séquence : rechercher un médicament et ses génériques . . . 15
2.5 MS3 · Stocks & Disponibilité . . . 15
2.5.1 Cas d'utilisation . . . 16
2.5.2 Diagramme de classes . . . 17
2.5.3 Diagramme de séquence : réapprovisionner un produit . . . 18
2.6 MS4 · Commandes & Retraits . . . 18
2.6.1 Cas d'utilisation . . . 19
2.6.2 Diagramme de classes . . . 20
2.6.3 Diagramme de séquence : passer une commande puis l'accepter ou la refuser . . . 21
2.7 MS5 · Notifications & Messagerie (microservice commun) . . . 22
2.7.1 Cas d'utilisation . . . 22
2.7.2 Diagramme de classes . . . 23
2.7.3 Diagramme de séquence : envoyer un message et consulter les notifications . . . 24

3 Conception technique **25**
3.1 Découpage et responsabilités des microservices . . . 25
3.2 Architecture globale . . . 26
3.3 Technologies . . . 27
3.4 Contrats des API REST . . . 27
3.4.1 Conventions communes . . . 27
3.4.2 MS1 · Pharmacies (port 8081, H2) . . . 27
3.4.3 MS2 · Médicaments (port 8082, H2) . . . 28
3.4.4 MS3 · Stocks & Disponibilité (port 8083, H2) . . . 28
3.4.5 MS4 · Commandes & Retraits (port 8084, MySQL) . . . 29
3.4.6 MS5 · Notifications & Messagerie (port 8085, MongoDB) . . . 29
3.5 Communication synchrone et asynchrone . . . 29
3.5.1 Communication synchrone (OpenFeign) . . . 30
3.5.2 Communication asynchrone (RabbitMQ) . . . 30
3.5.3 Vue des interactions par microservice . . . 30
3.6 Sécurité et résilience . . . 32
3.6.1 Sécurité . . . 32
3.6.2 Résilience et cohérence . . . 32

4 Organisation et perspectives **33**
4.1 Répartition des tâches . . . 33
4.2 Organisation du dépôt Git et gestion de projet . . . 33
4.3 Planification du projet . . . 34
4.4 Risques et mesures . . . 34

Conclusion et prochaines étapes **35**

A Exemples d'échanges **36**
A.1 Passer une commande . . . 36
A.2 Erreur : stock insuffisant . . . 36
A.3 Événement `stock.replenished` . . . 36
A.4 Événement `order.events` . . . 36

Table des figures

2.1 Diagramme global de cas d'utilisation 9
2.2 Diagramme de classes global, organisé par microservice 10
2.3 MS1 – Diagramme de cas d'utilisation 11
2.4 MS1 – Diagramme de classes 11
2.5 MS1 – Créer puis valider une pharmacie 12
2.6 MS2 – Diagramme de cas d'utilisation 13
2.7 MS2 – Diagramme de classes 14
2.8 MS2 – Rechercher un médicament et ses génériques 15
2.9 MS3 – Diagramme de cas d'utilisation 16
2.10 MS3 – Diagramme de classes 17
2.11 MS3 – Réapprovisionner un produit 18
2.12 MS4 – Diagramme de cas d'utilisation 19
2.13 MS4 – Diagramme de classes 20
2.14 MS4 – Passer une commande puis l'accepter ou la refuser 21
2.15 MS5 – Diagramme de cas d'utilisation 22
2.16 MS5 – Diagramme de classes 23
2.17 MS5 – Envoyer un message et consulter les notifications 24

3.1 Architecture technique globale 26
3.2 Interactions de MS1 (gauche) et MS2 (droite) 31
3.3 Interactions de MS3 (gauche) et MS4 (droite) 31
3.4 Interactions de MS5 31

Liste des tableaux

1.1 Objectifs du projet 7
1.2 Acteurs et besoins fonctionnels 7
1.3 Besoins non fonctionnels 8
1.4 Composition de l'équipe 8

2.1 Transitions du statut d'une commande 21

3.1 Découpage en microservices 25
3.2 Composants de l'architecture 26
3.3 Technologies retenues 27
3.4 Appels synchrones entre microservices 30
3.5 Événements métier 30
3.6 Mécanismes de résilience et de cohérence 32

4.1 Répartition des tâches 33
4.2 Planning prévisionnel 34
4.3 Risques identifiés et mesures prévues 34

Introduction générale

Trouver un médicament reste une démarche pénible pour de nombreux patients : la disponibilité varie d'une officine à l'autre, l'information n'est pas centralisée et le patient multiplie appels et déplacements. Dans le cadre du module *Applications Web Distribuées*, notre équipe propose **MedFinder**, une plateforme web qui relie patients, pharmaciens et administrateurs autour d'un objectif simple : savoir rapidement où trouver un médicament, le commander sans risque de survente et être prévenu dès son retour en stock.

Ce rapport présente la conception de MedFinder soumise à la première validation. Il reprend la structure de notre présentation et la détaille :

— la **présentation du projet** : contexte, problématique, solution, objectifs, acteurs et besoins (chapitre 1) ;

— la **conception fonctionnelle** : cas d'utilisation, diagrammes de classes et de séquence, globalement puis pour chacun des cinq microservices (chapitre 2) ;

— la **conception technique** : découpage, architecture, technologies, contrats d'API REST, communication synchrone et asynchrone, sécurité et résilience (chapitre 3) ;

— l'**organisation et les perspectives** : répartition des tâches, gestion du dépôt Git, planification, risques et prochaines étapes (chapitre 4).

CHAPITRE 1

Présentation du projet

1.1 Contexte

La disponibilité des médicaments varie fortement d'une pharmacie à l'autre, en particulier pour les produits en tension. Aujourd'hui :

— un même médicament peut être disponible dans une pharmacie et en rupture dans la pharmacie voisine ;

— le patient téléphone ou se déplace de pharmacie en pharmacie sans savoir laquelle dispose du produit ;

— les pharmaciens reçoivent de nombreuses demandes de disponibilité et n'ont pas d'outil simple pour publier leur stock ou prévenir les patients d'un réapprovisionnement.

1.2 Problématique

Comment permettre aux patients de **trouver rapidement les médicaments disponibles à proximité** et d'**être informés des retours en stock** ?

1.3 Solution proposée

MedFinder est une plateforme web qui facilite la recherche et la réservation de médicaments auprès des pharmacies partenaires. Elle permet aux patients de :

— rechercher un médicament (par nom ou par molécule active) ;

— localiser les pharmacies proches et les pharmacies de garde ;

— consulter la disponibilité et le prix des médicaments dans chaque pharmacie ;

— réserver un médicament en passant une commande, puis suivre son état jusqu'au retrait ;

— recevoir des alertes en cas de réapprovisionnement et échanger avec le pharmacien.

Les pharmaciens gèrent leur pharmacie, leur stock et les commandes reçues ; l'administrateur valide les pharmacies, gère le catalogue et suit l'activité de la plateforme.

1.4 Objectifs

Objectif	Ce que cela signifie
Localisation rapide des médicaments	Trouver en quelques secondes une pharmacie proche qui dispose du médicament recherché
Réservation sécurisée sans survente	Deux patients ne peuvent jamais obtenir la dernière boîte : le stock est décrémenté de façon atomique
Alertes automatiques de disponibilité	Le patient est prévenu automatiquement lorsque le médicament revient en stock
Architecture microservices sécurisée et résiliente	Services indépendants, authentification centralisée, tolérance aux pannes, déploiement conteneurisé

Table 1.1 – Objectifs du projet

1.5 Acteurs et besoins fonctionnels

MedFinder distingue trois acteurs humains, chacun associé à un rôle Keycloak.

Acteur	Besoins fonctionnels	Rôle Keycloak
Patient	Rechercher un médicament ; localiser les pharmacies ; consulter les stocks et les prix ; passer et suivre une commande ; créer des alertes ; communiquer avec le pharmacien	`PATIENT`
Pharmacien	Gérer sa pharmacie ; gérer les stocks et les prix ; accepter ou refuser les commandes ; gérer les alertes ; répondre aux messages	`PHARMACIST`
Administrateur	Valider ou désactiver les pharmacies ; gérer le catalogue des médicaments et les catégories ; consulter les statistiques ; suivre le journal d'activité	`ADMIN`

Table 1.2 – Acteurs et besoins fonctionnels

1.6 Besoins non fonctionnels

Besoin	Réponse apportée
Sécurité	Keycloak (OpenID Connect, jetons JWT) et contrôle d'accès par rôle et par propriétaire de la ressource
Disponibilité	Circuit breaker Resilience4j, timeouts et mode dégradé ; files RabbitMQ durables
Cohérence	Décrément atomique du stock et compensation en cas de refus ou d'annulation
Performance	Recherche rapide grâce à des services dédiés et à des requêtes filtrées
Portabilité	Chaque composant dans un conteneur, lancement complet avec Docker Compose
Maintenabilité	Contrats d'API documentés (OpenAPI, Swagger UI), configuration centralisée

Table 1.3 – Besoins non fonctionnels

1.7 Équipe

Membre	Rôle dans l'équipe	Microservice individuel
Ferdaouas Oueslati	Présentatrice principale	MS3 · Stocks & Disponibilité
Abdelhamid Ayadi	Responsable storytelling	MS2 · Médicaments
Jihed Khlifi	Chef de projet	MS1 · Pharmacies
Insaf Makhloufi	Responsable contenu	MS4 · Commandes & Retraits

Table 1.4 – Composition de l'équipe

Le microservice commun MS5 (Notifications & Messagerie) est conçu et développé par toute l'équipe.

CHAPITRE 2

Conception fonctionnelle

2.1 Diagramme global de cas d'utilisation

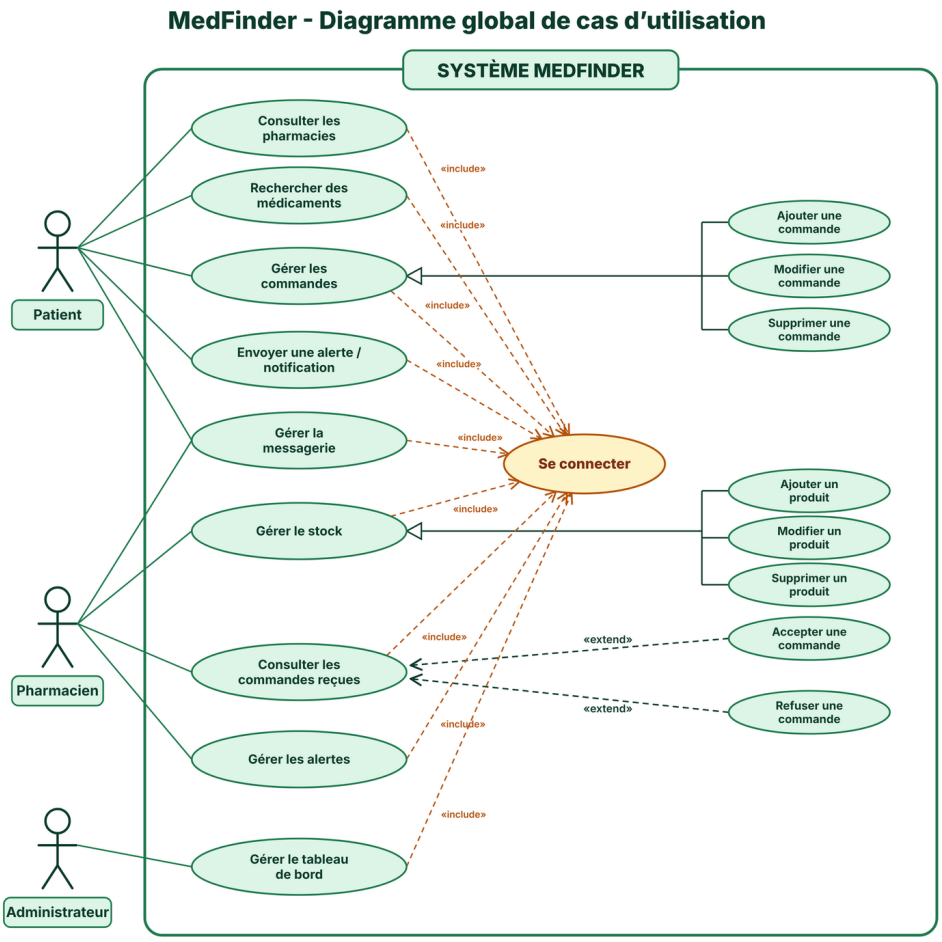


Figure 2.1 – Diagramme global de cas d'utilisation

Le diagramme global présente les trois acteurs et les fonctionnalités de la plateforme :

- le **patient** consulte les pharmacies, recherche des médicaments, gère ses commandes, envoie des alertes et utilise la messagerie ;
- le **pharmacien** gère son stock, consulte les commandes reçues et les accepte ou les refuse, gère les alertes et utilise la messagerie ;
- l'**administrateur** gère le tableau de bord de la plateforme.

Toutes les actions incluent le cas « Se connecter » (relation `«include»`) : l'authentification est obligatoire. « Gérer les commandes » et « Gérer le stock » sont spécialisés (généralisation) en ajout, modification et suppression ; « Accepter » et « Refuser une commande » étendent (`«extend»`) la consultation des commandes reçues.

2.2 Diagramme de classes global

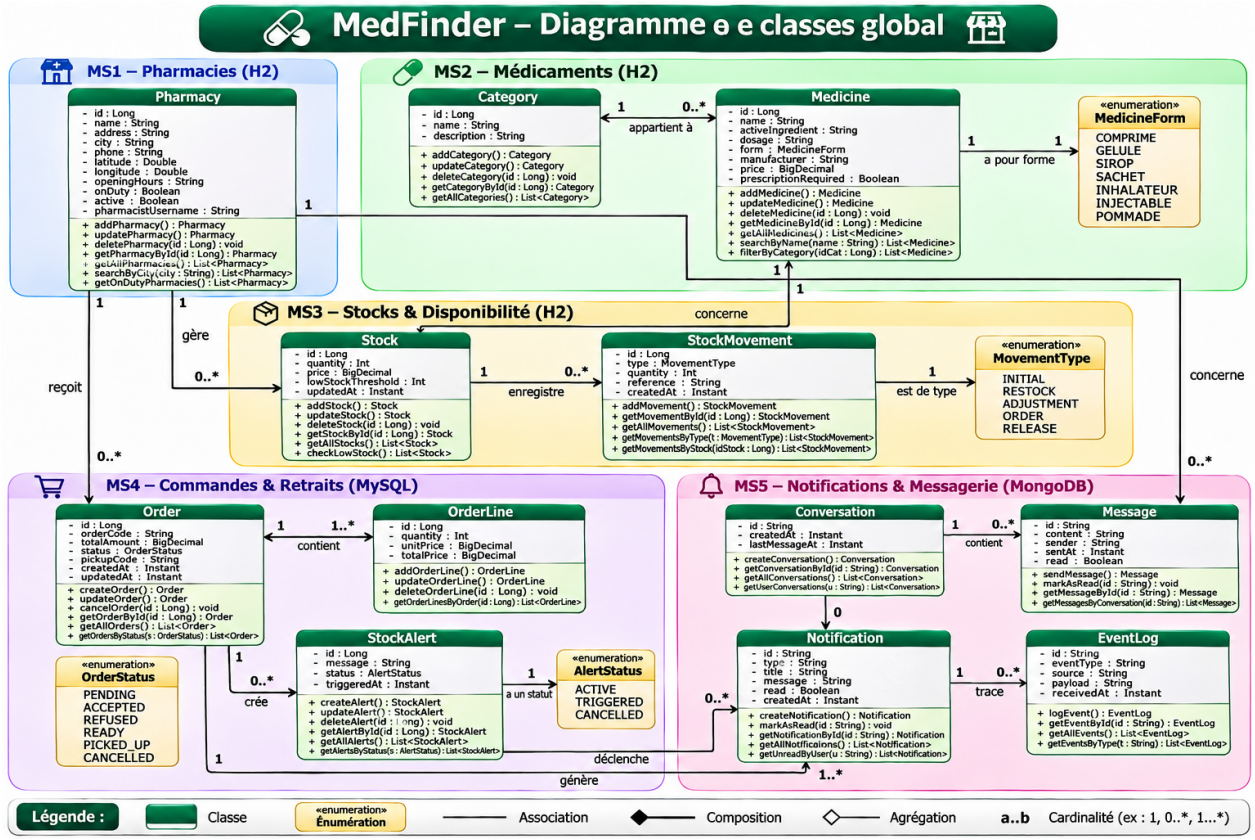


Figure 2.2 – Diagramme de classes global, organisé par microservice

Le diagramme global regroupe les classes par microservice, chacun avec sa propre base. Les associations entre deux microservices (par exemple une pharmacie et ses stocks, ou une commande et les notifications qu'elle génère) sont des **associations logiques** : elles sont réalisées par identifiant (`pharmacyId`, `medicineId`), jamais par une clé étrangère ou une jointure entre bases (principe *database per service*).

2.3 MS1 · Pharmacies

2.3.1 Cas d'utilisation

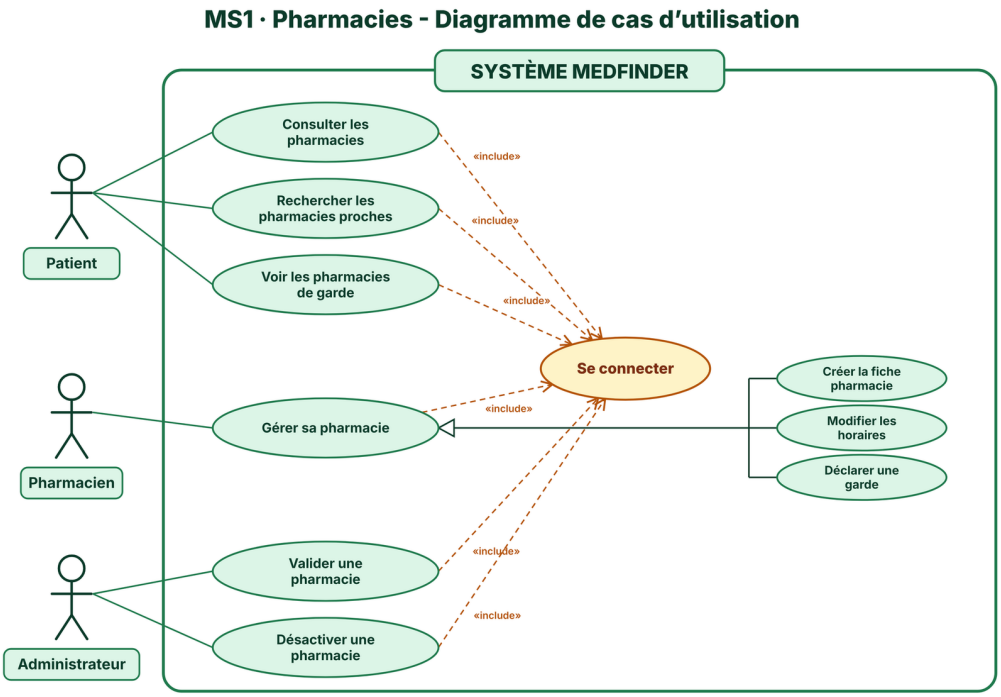


Figure 2.3 – MS1 – Diagramme de cas d'utilisation

Le patient consulte les pharmacies, recherche les plus proches et voit les pharmacies de garde. Le pharmacien gère sa pharmacie : il crée sa fiche, modifie ses horaires et déclare ses gardes. L'administrateur valide ou désactive les pharmacies ; une pharmacie n'est visible des patients qu'après validation.

2.3.2 Diagramme de classes

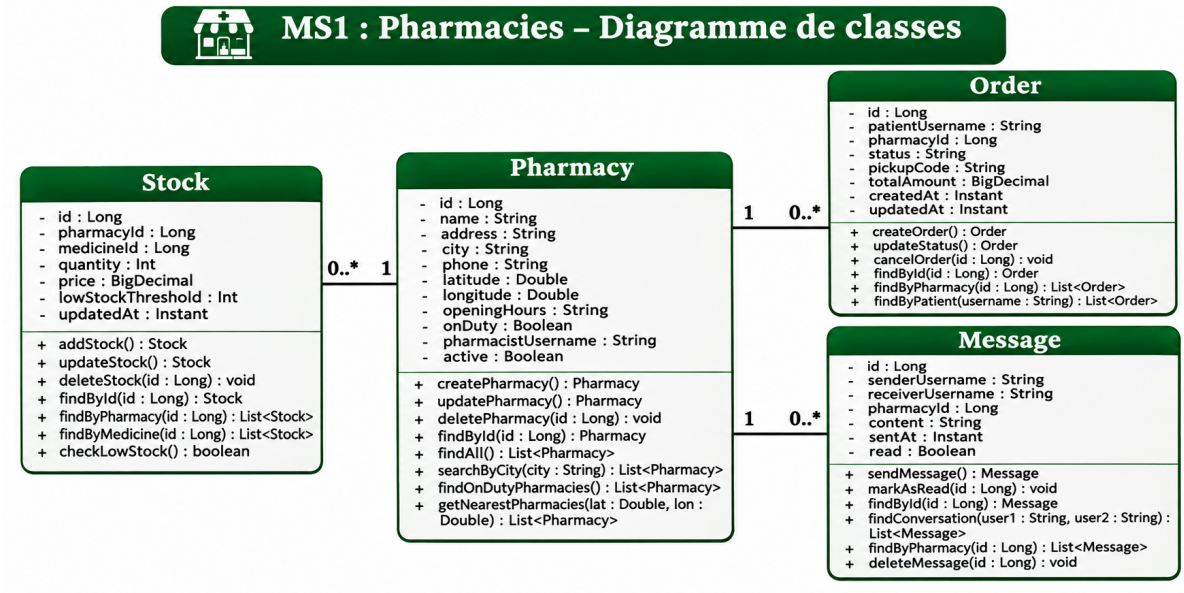


Figure 2.4 – MS1 – Diagramme de classes

La classe centrale `Pharmacy` porte l'adresse, les coordonnées GPS (`latitude`, `longitude`) pour la recherche par proximité, les horaires, l'indicateur de garde (`onDuty`), le compte du pharmacien rattaché et le statut `active`. Les classes `Stock`, `Order` et `Message` appartiennent respectivement à MS3, MS4 et MS5 : elles apparaissent pour montrer qu'elles référencent une pharmacie par son identifiant.

2.3.3 Diagramme de séquence : créer puis valider une pharmacie

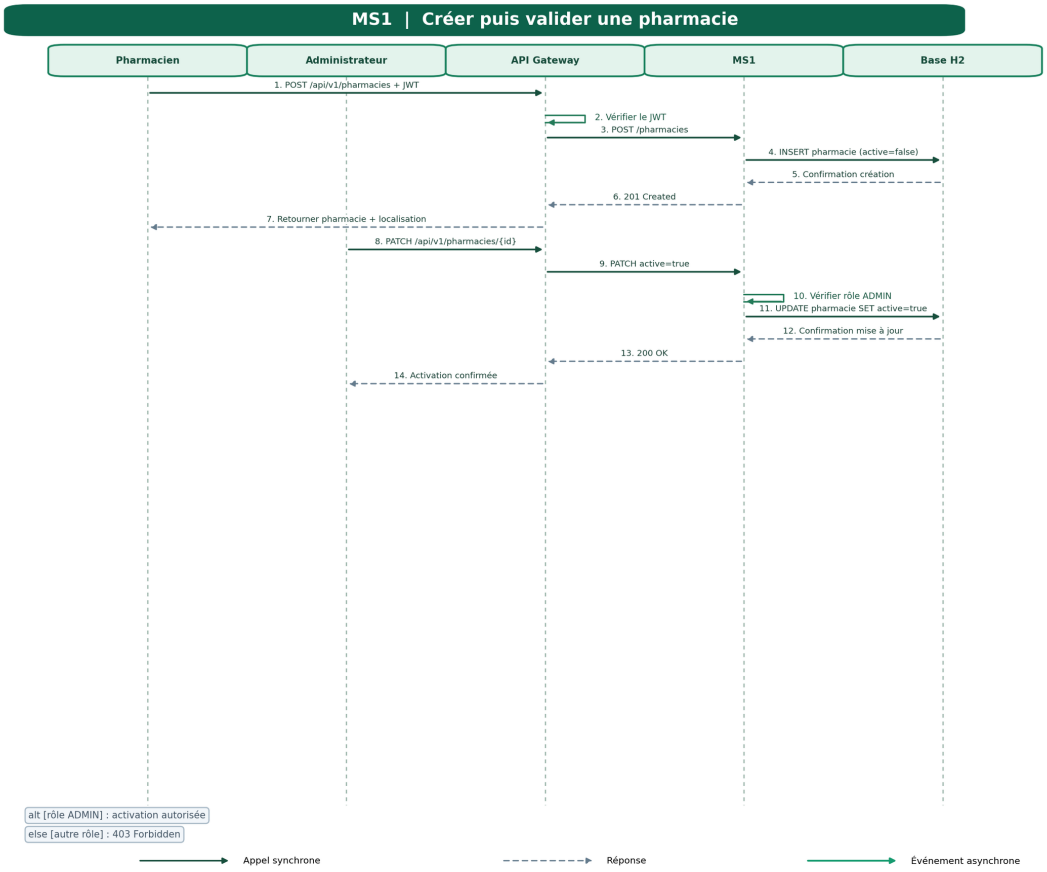


Figure 2.5 – MS1 – Créer puis valider une pharmacie

Le pharmacien envoie la fiche via la Gateway, qui vérifie son jeton. MS1 l'enregistre **inactive** et répond `201 Created`. L'administrateur la valide ensuite par un `PATCH` : MS1 vérifie le rôle `ADMIN`, passe la pharmacie à l'état actif et répond `200` ; tout autre rôle reçoit `403 Forbidden`.

2.4 MS2 · Médicaments

2.4.1 Cas d'utilisation

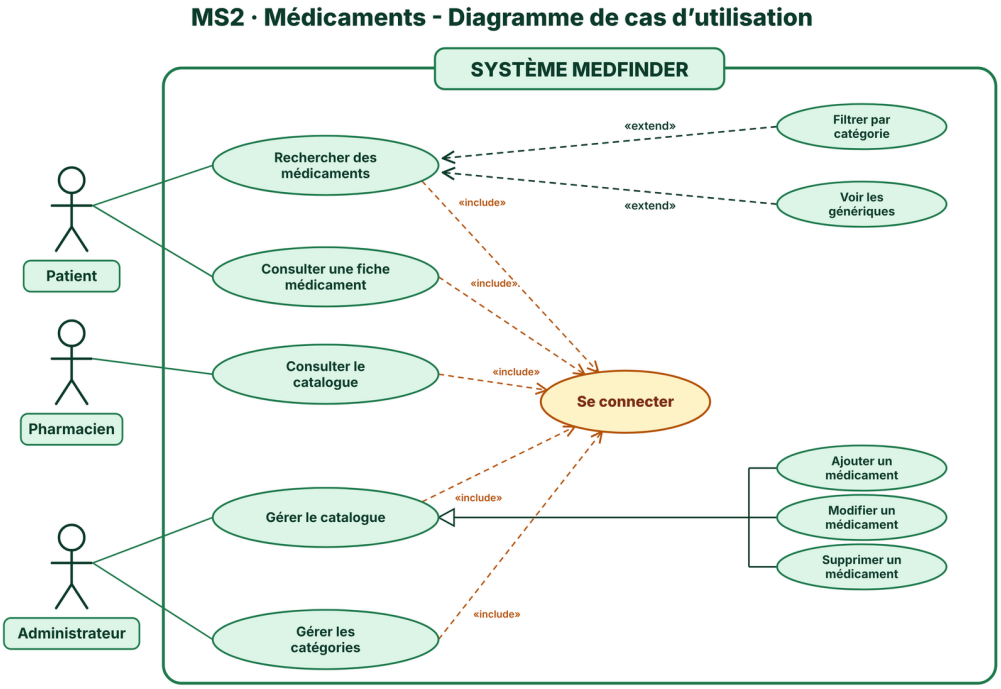


Figure 2.6 – MS2 – Diagramme de cas d'utilisation

Le patient recherche des médicaments ; la recherche peut être étendue par un filtre de catégorie ou par l'affichage des génériques (`«extend»`). Il consulte aussi la fiche d'un médicament. Le pharmacien consulte le catalogue ; l'administrateur le gère (ajout, modification, suppression) ainsi que les catégories.

2.4.2 Diagramme de classes

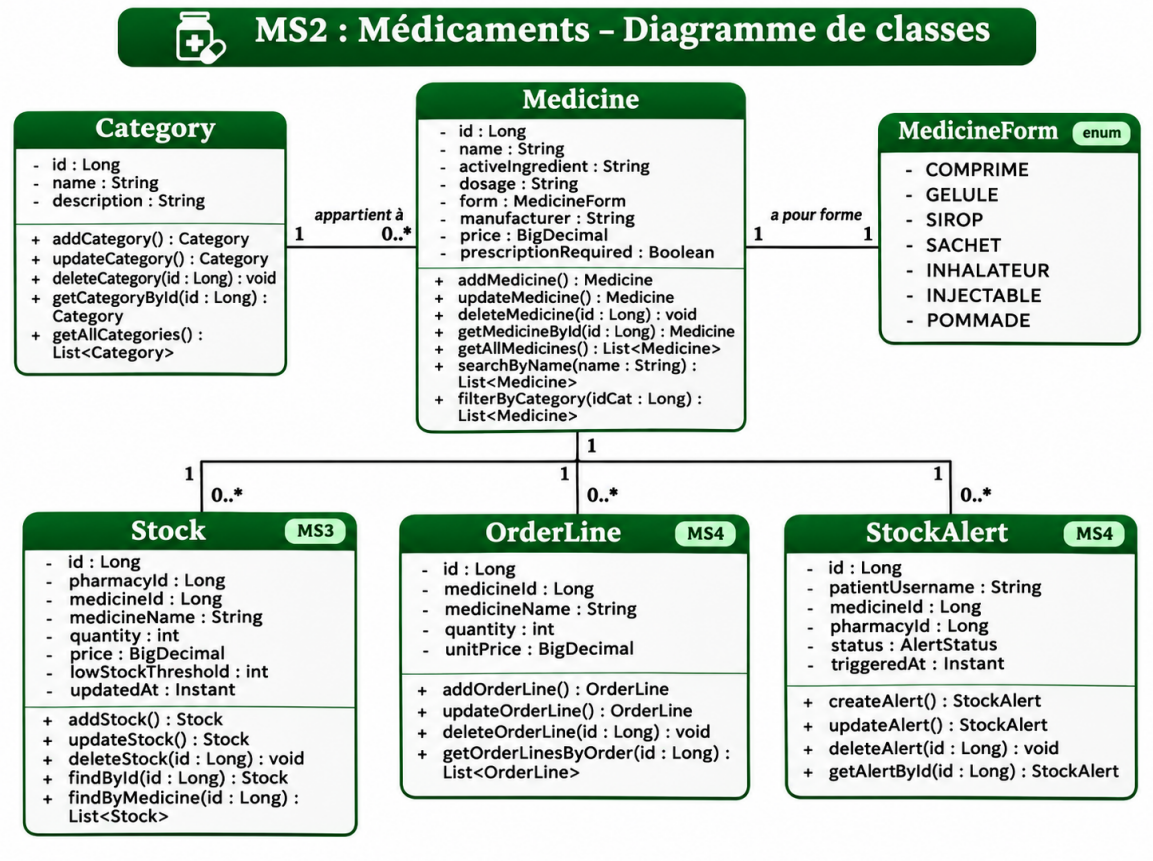


Figure 2.7 – MS2 – Diagramme de classes

Une `Category` regroupe plusieurs `Medicine`. Un médicament est décrit par son nom, sa molécule active (`activeIngredient`), son dosage, sa forme (énumération `MedicineForm` : comprimé, gélule, sirop, sachet, inhalateur, injectable, pommade), son fabricant, un prix indicatif et l'indication d'ordonnance. Les génériques sont les médicaments de même molécule active.

2.4.3 Diagramme de séquence : rechercher un médicament et ses génériques

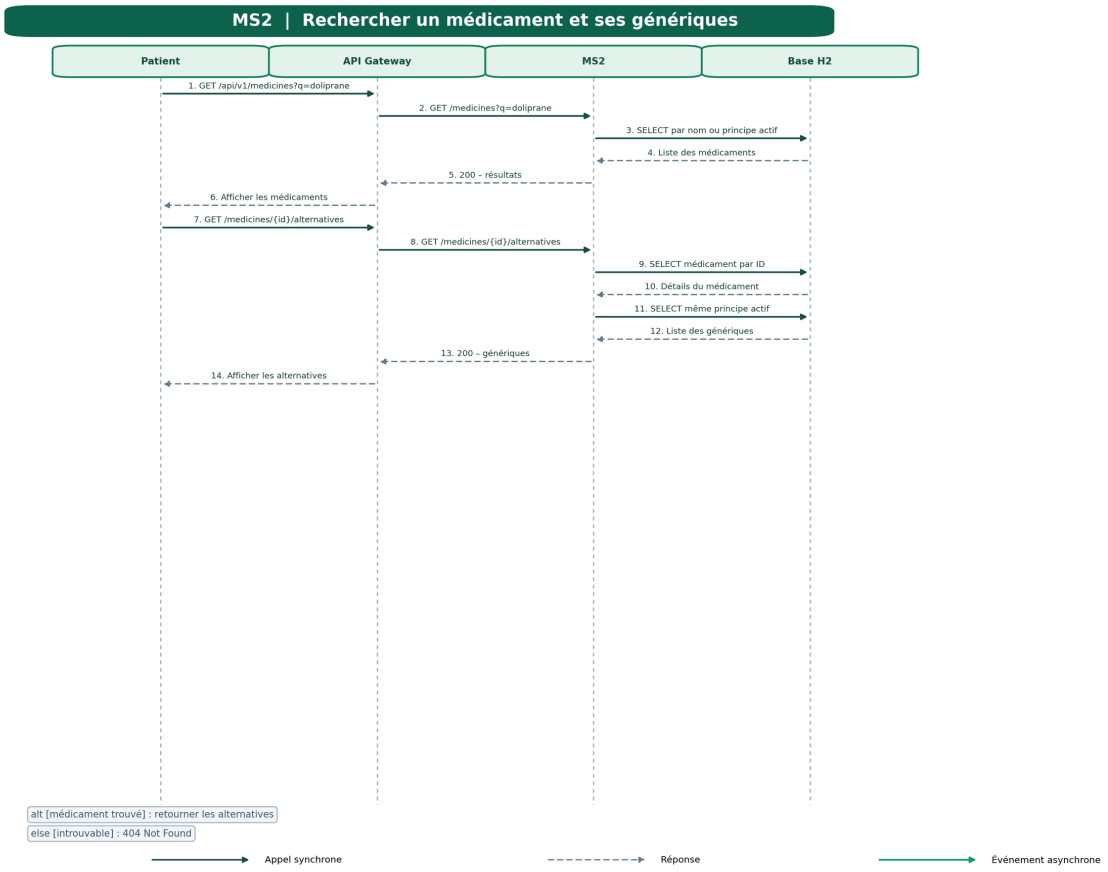


Figure 2.8 – MS2 – Rechercher un médicament et ses génériques

La recherche passe par une route publique de la Gateway. MS2 interroge sa base par nom ou par molécule et renvoie une page de résultats (`200`). La demande des génériques d'un médicament renvoie la liste des médicaments de même molécule, ou `404` au format *Problem Details* si l'identifiant est inconnu.

2.5 MS3 · Stocks & Disponibilité

2.5.1 Cas d'utilisation

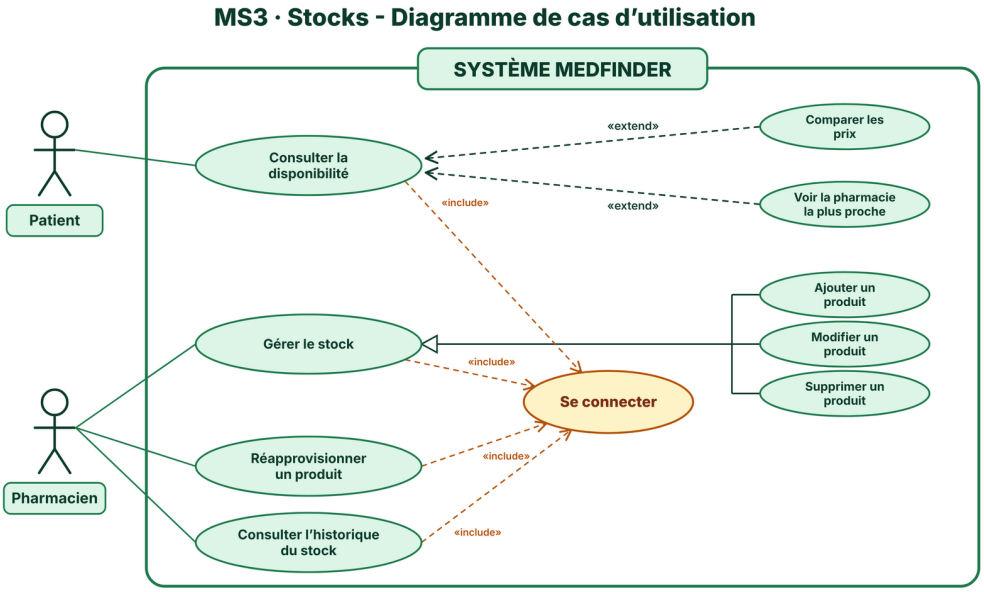


Figure 2.9 – MS3 – Diagramme de cas d'utilisation

Le patient consulte la disponibilité d'un médicament, avec comparaison des prix et repérage de la pharmacie la plus proche (`«extend»`). Le pharmacien gère le stock de sa pharmacie (ajout, modification, suppression d'un produit), le réapprovisionne et consulte l'historique des mouvements.

2.5.2 Diagramme de classes

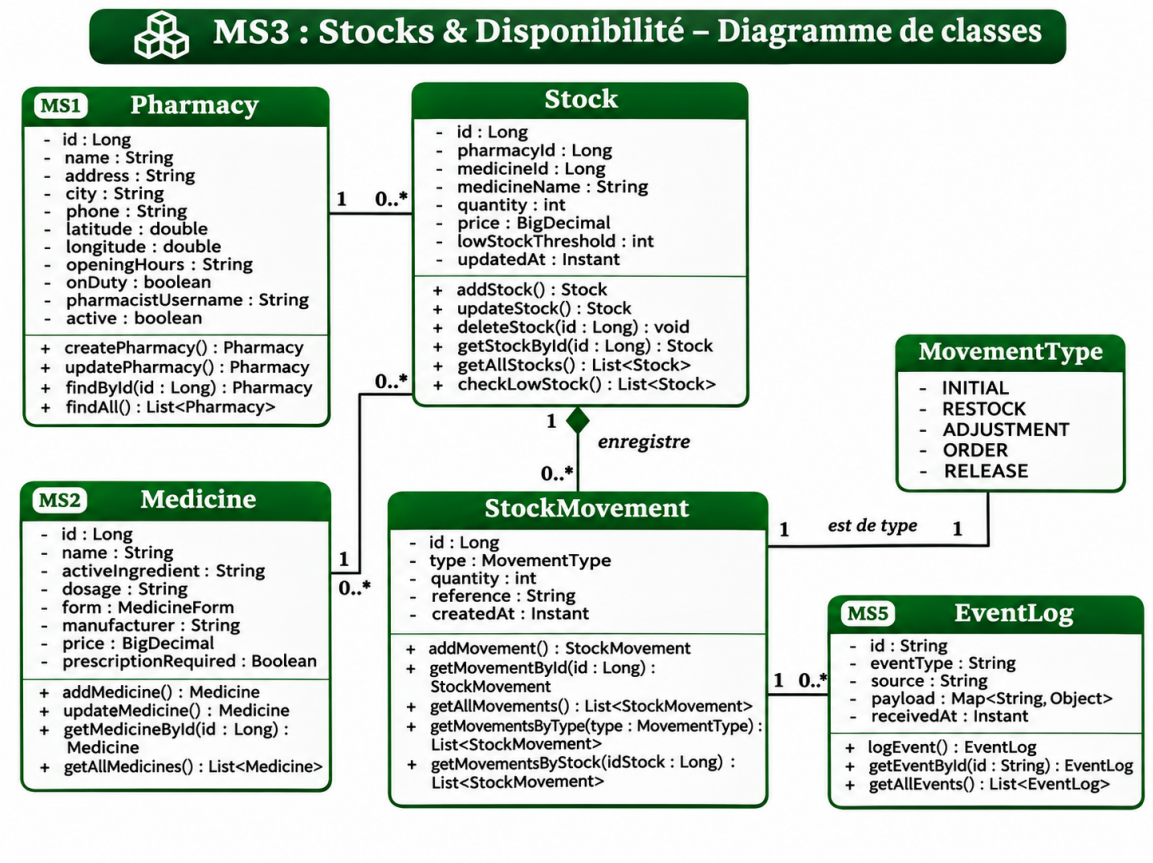


Figure 2.10 – MS3 – Diagramme de classes

Un `Stock` associe une pharmacie et un médicament (références par identifiant) à une quantité, un prix et un seuil d'alerte. Chaque modification est tracée par un `StockMovement` dont le type (`MovementType`) vaut `INITIAL`, `RESTOCK`, `ADJUSTMENT`, `ORDER` ou `RELEASE`. Le couple (pharmacie, médicament) est unique.

2.5.3 Diagramme de séquence : réapprovisionner un produit

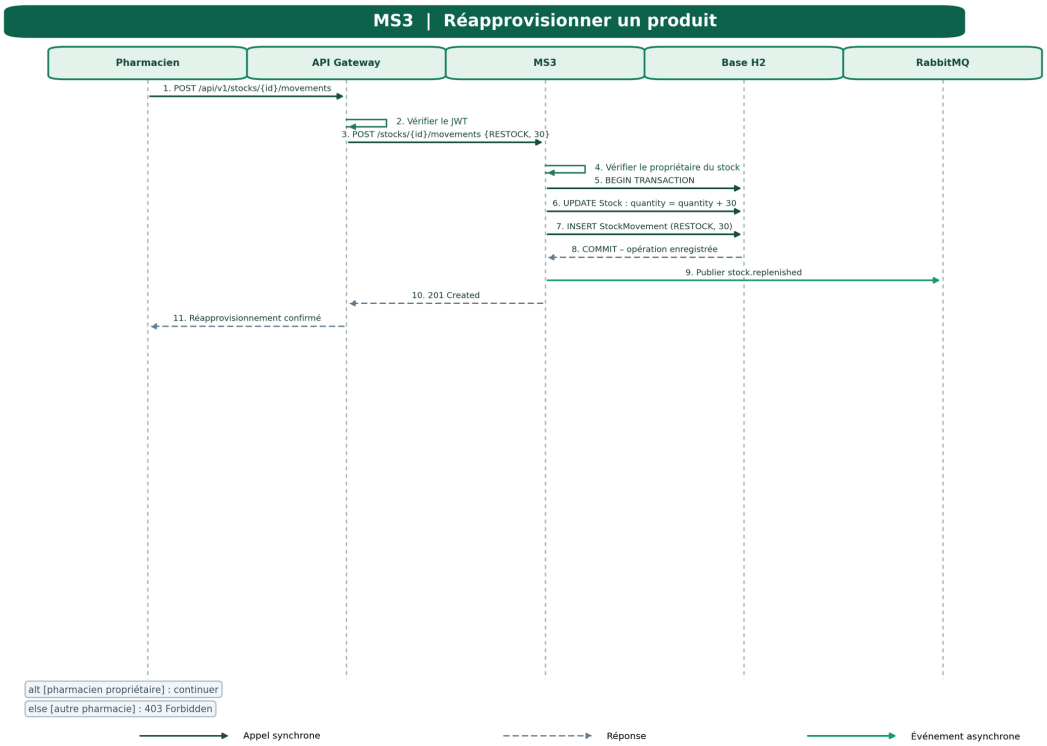


Figure 2.11 – MS3 – Réapprovisionner un produit

Si le pharmacien est bien propriétaire du stock, MS3 augmente la quantité, enregistre le mouvement `RESTOCK`, publie l'événement `stock.replenished` sur RabbitMQ et répond `201`. L'événement est livré à MS4 (déclenchement des alertes) et à MS5 (notifications). Un pharmacien d'une autre pharmacie reçoit `403`.

2.6 MS4 · Commandes & Retraits

2.6.1 Cas d'utilisation

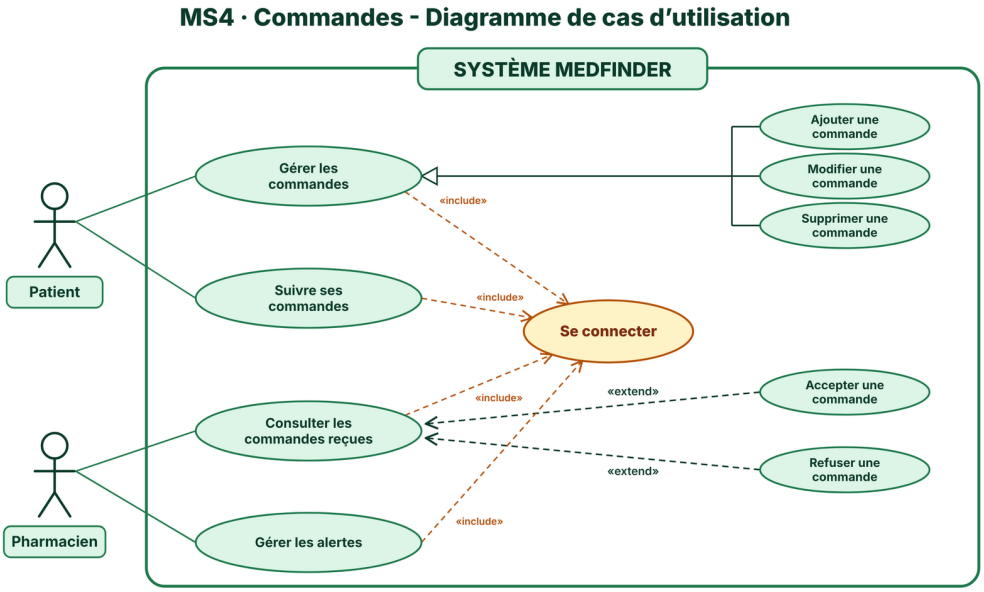


Figure 2.12 – MS4 – Diagramme de cas d'utilisation

Le patient gère ses commandes (ajout, modification et suppression tant qu'elles sont en attente) et les suit. Le pharmacien consulte les commandes reçues, peut les accepter ou les refuser (`«extend»`), et gère les alertes de retour en stock.

2.6.2 Diagramme de classes

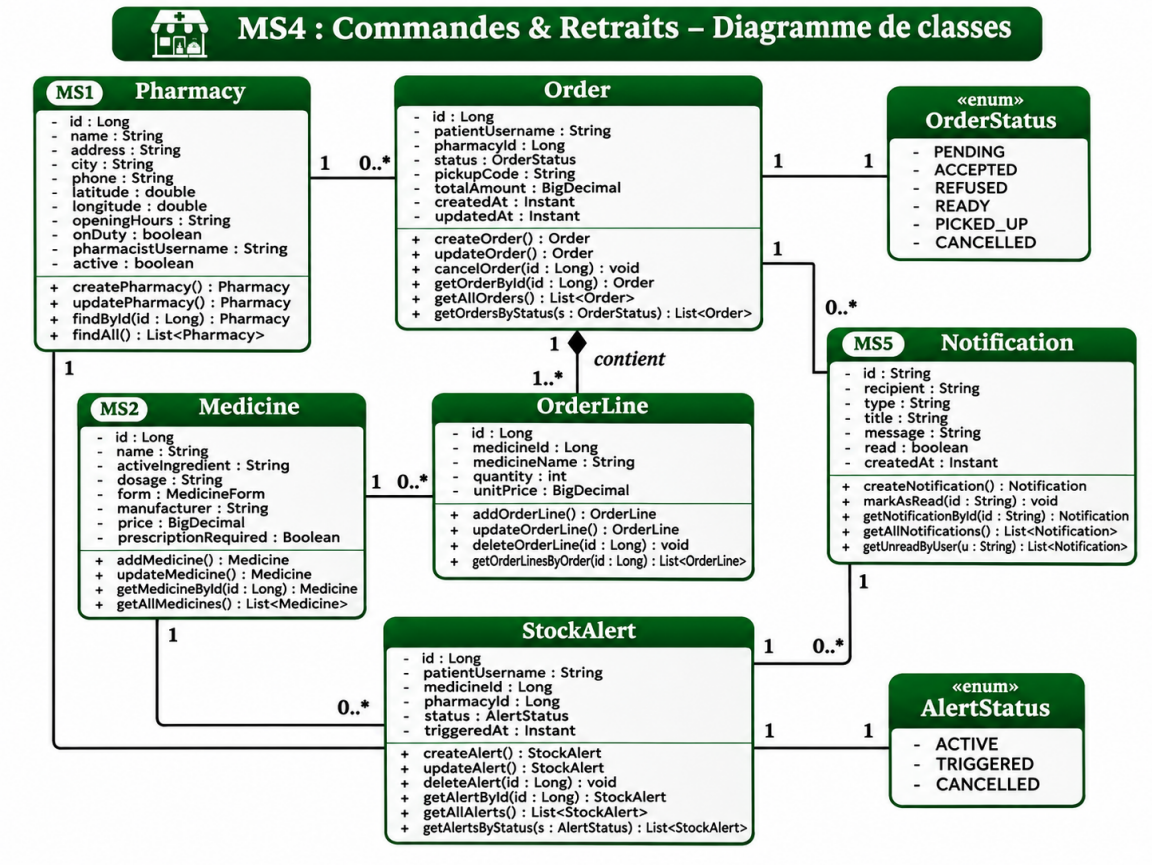


Figure 2.13 – MS4 – Diagramme de classes

Une `Order` contient une ou plusieurs `OrderLine` (médicament, quantité, prix unitaire) et porte un code de retrait, un montant total et un statut `OrderStatus` : `PENDING`, `ACCEPTED`, `REFUSED`, `READY`, `PICKED_UP`, `CANCELLED`. Une `StockAlert` mémorise la demande d'un patient d'être prévenu du retour d'un médicament (statut `ACTIVE`, `TRIGGERED` ou `CANCELLED`).

2.6.3 Diagramme de séquence : passer une commande puis l'accepter ou la refuser

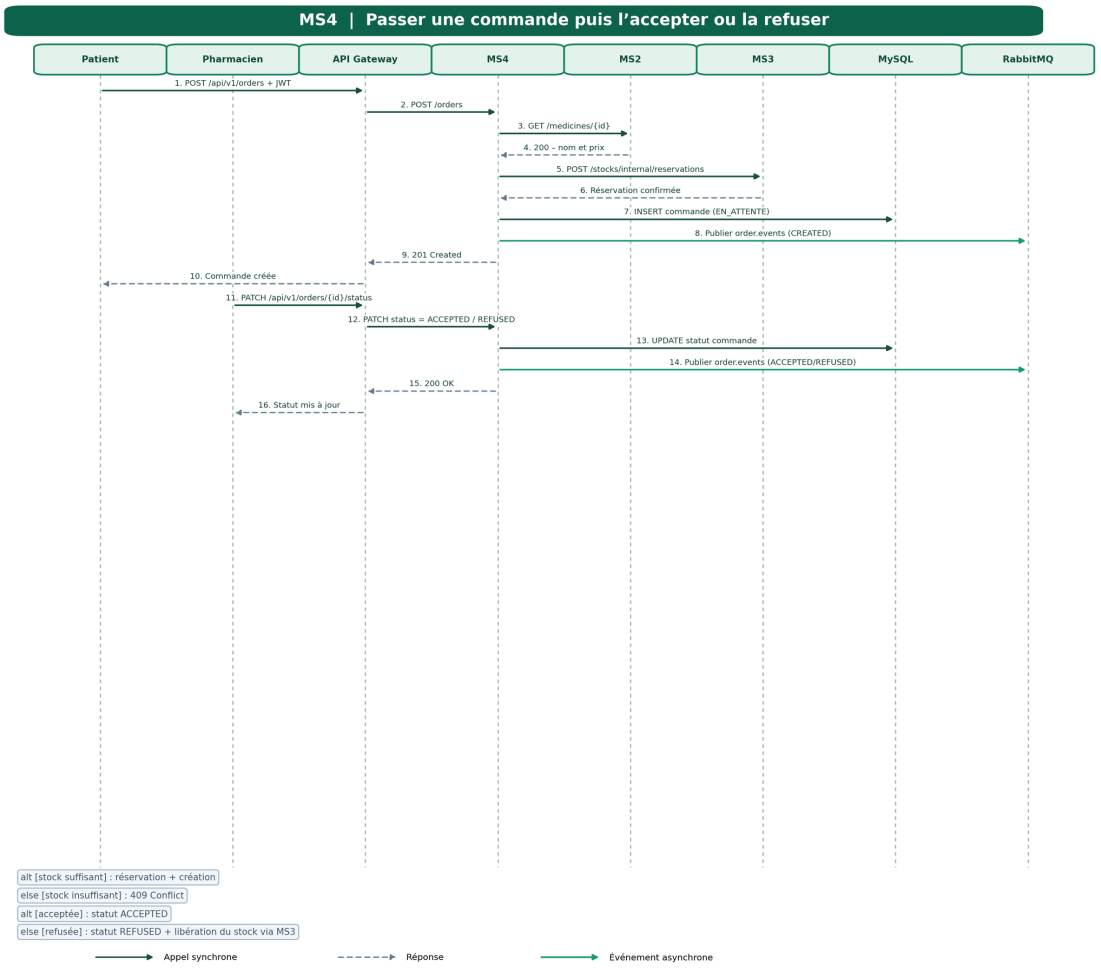


Figure 2.14 – MS4 – Passer une commande puis l'accepter ou la refuser

MS4 appelle MS2 (nom et prix du médicament) puis MS3, qui réserve les quantités de façon atomique. Si le stock suffit, la commande est enregistrée dans MySQL avec le statut `PENDING`, l'événement `order.events` est publié et le patient reçoit `201`. Sinon MS3 répond `409` et rien n'est créé. Le pharmacien accepte ou refuse ensuite la commande ; un refus publie `stock.released` pour rendre les quantités à MS3.

Cycle de vie d'une commande

De	Vers	Déclencheur	Acteur
`PENDING`	`ACCEPTED`	Acceptation de la commande	Pharmacien
`PENDING`	`REFUSED`	Refus ; les quantités sont rendues	Pharmacien
`PENDING`	`CANCELLED`	Annulation ; les quantités sont rendues	Patient
`ACCEPTED`	`READY`	Commande préparée	Pharmacien
`READY`	`PICKED_UP`	Retrait avec le code	Pharmacien

Table 2.1 – Transitions du statut d'une commande

2.7 MS5 · Notifications & Messagerie (microservice commun)

2.7.1 Cas d'utilisation

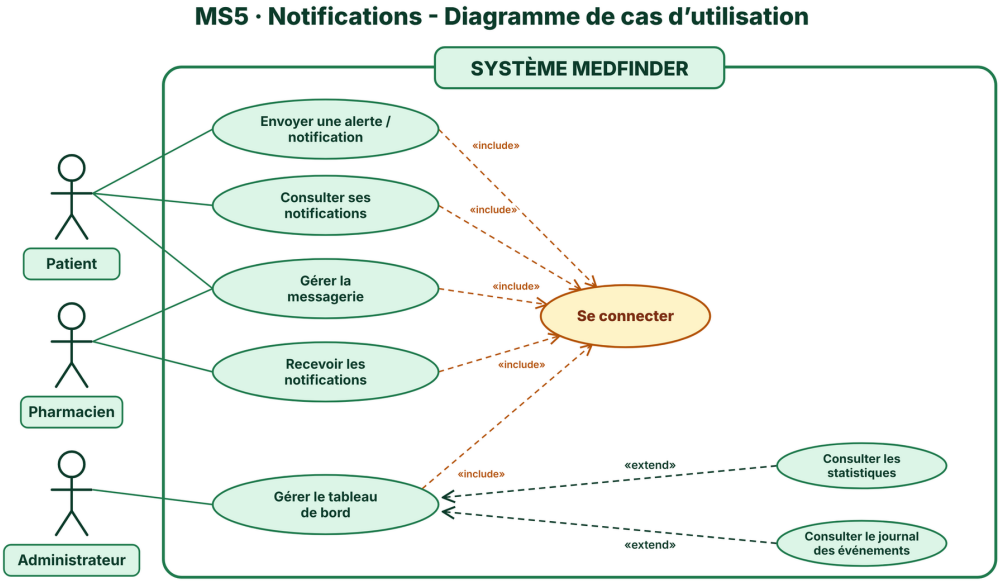


Figure 2.15 – MS5 – Diagramme de cas d'utilisation

Le patient envoie des alertes et consulte ses notifications ; le pharmacien reçoit les notifications ; la messagerie est partagée entre patient et pharmacien. L'administrateur gère le tableau de bord, étendu par la consultation des statistiques et du journal des événements.

2.7.2 Diagramme de classes

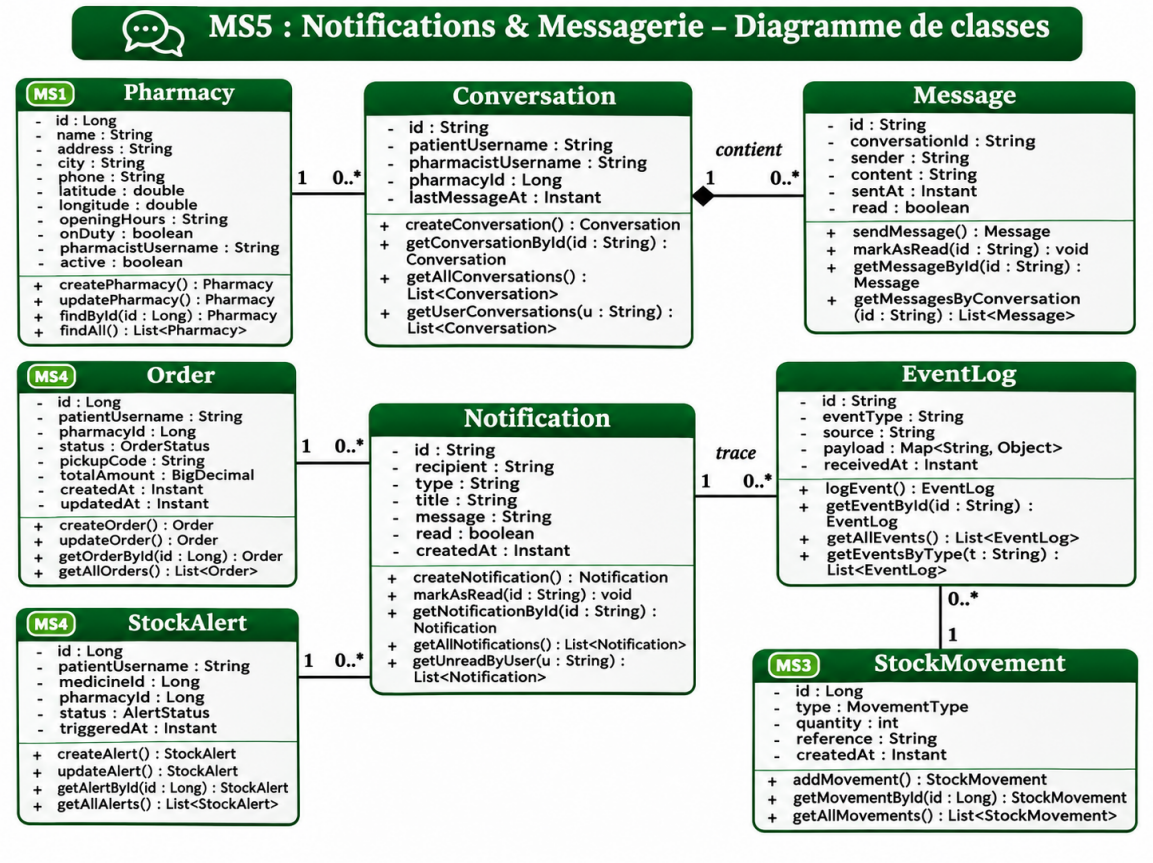


Figure 2.16 – MS5 – Diagramme de classes

MS5 stocke des documents MongoDB : une `Conversation` entre un patient et un pharmacien contient des `Message` ; une `Notification` est adressée à un destinataire ; un `EventLog` trace chaque événement reçu et alimente les statistiques du tableau de bord. Les classes marquées MS1, MS3 et MS4 sont les sources des événements et des références.

2.7.3 Diagramme de séquence : envoyer un message et consulter les notifications

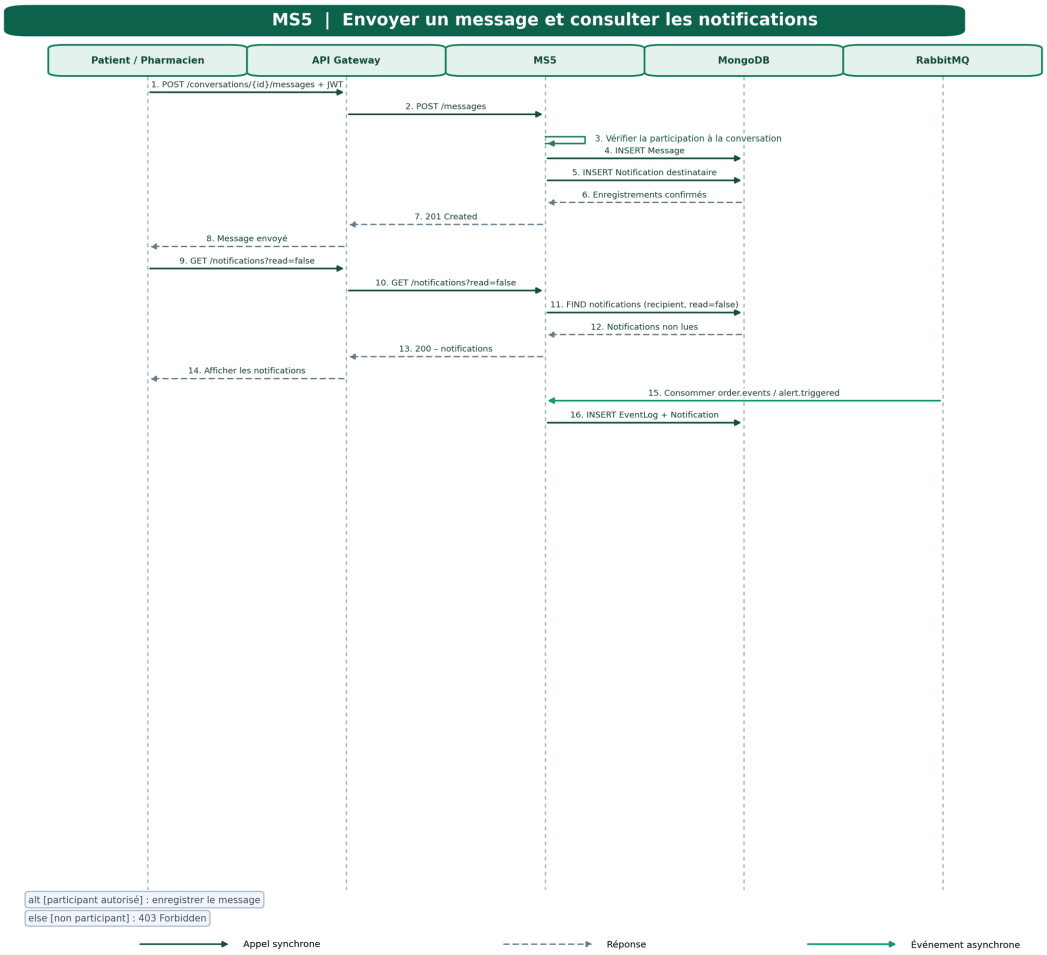


Figure 2.17 – MS5 – Envoyer un message et consulter les notifications

Lorsqu'un participant envoie un message, MS5 l'enregistre et crée une notification pour le destinataire. En parallèle, MS5 consomme les événements RabbitMQ (`order.events`, `alert.triggered`, `stock.replenished`) : il journalise chaque événement et crée la notification correspondante. Si MS5 est arrêté, les messages attendent dans les files et sont traités au redémarrage.

CHAPITRE 3

Conception technique

3.1 Découpage et responsabilités des microservices

Le découpage suit les **contextes métier** du domaine : chaque microservice correspond à une capacité cohérente, possède ses propres données et peut évoluer, être déployé ou tomber en panne indépendamment des autres.

Microservice	Responsabilités	Base	Responsable
MS1 · Pharmacies	Fiches des pharmacies, informations, localisation, horaires, gardes, validation	H2	Jihed Khlifi
MS2 · Médicaments	Catalogue des médicaments et catégories, recherche, génériques	H2	Abdelhamid Ayadi
MS3 · Stocks & Disponibilité	Quantités, disponibilités, prix, réapprovisionnement, historique, réservation des quantités	H2	Ferdaouas Oueslati
MS4 · Commandes & Retraits	Commandes, acceptations et refus, retraits, alertes de retour en stock	MySQL	Insaf Makhloufi
MS5 · Notifications & Messagerie	Notifications, messagerie, journal des événements, tableau de bord	MongoDB	Toute l'équipe

Table 3.1 – Découpage en microservices

Choix des bases de données

— **H2 pour MS1, MS2 et MS3** : base imposée pour les microservices individuels ; données relationnelles simples, et mises à jour atomiques en SQL pour le stock.

— **MySQL pour MS4** : les commandes sont des données transactionnelles qui doivent survivre à un redémarrage.

— **MongoDB pour MS5** : notifications, messages et événements ont des structures variées (documents) ; les statistiques sont calculées par agrégation.

3.2 Architecture globale

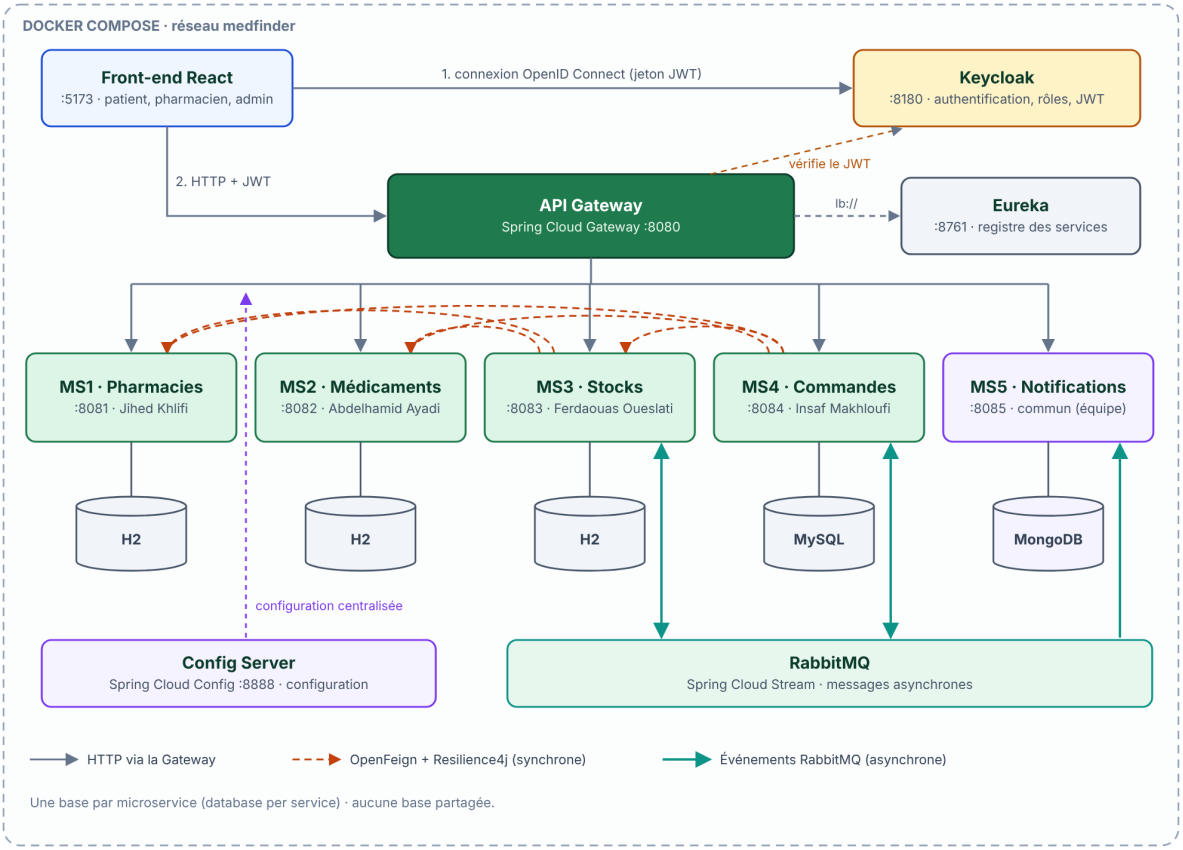


Figure 3.1 – Architecture technique globale

Le front-end React n'appelle qu'un seul point d'entrée, l'**API Gateway**. Celle-ci vérifie le jeton JWT délivré par **Keycloak**, puis route la requête vers le microservice concerné, dont elle obtient l'adresse auprès d'**Eureka**. Au démarrage, chaque service récupère sa configuration auprès du **Config Server**. Les microservices s'appellent directement entre eux avec OpenFeign (synchrone) et échangent des événements par **RabbitMQ** (asynchrone). Chaque microservice possède sa propre base ; aucune base n'est partagée.

Composant	Rôle	Port
Front-end React	Interface patient, pharmacien et administrateur	5173
API Gateway	Point d'entrée unique, routage `lb://`, validation du JWT, CORS	8080
Eureka Server	Registre et découverte des services	8761
Config Server	Configuration centralisée (ports, bases, RabbitMQ, Eureka)	8888
Keycloak	Comptes, rôles, émission des jetons	8180
RabbitMQ	Échanges et files durables des événements	5672
MS1 à MS5	Microservices métier, documentés par Swagger UI	8081 – 8085

Table 3.2 – Composants de l'architecture

3.3 Technologies

Besoin	Technologie	Usage
Microservices	Spring Boot 3, Java 17, Maven	Un projet par microservice
Point d'entrée	Spring Cloud Gateway	Routage et sécurité en entrée
Découverte	Netflix Eureka	Localisation dynamique des services
Configuration	Spring Cloud Config Server	Configuration centralisée
Sécurité	Keycloak, Spring Security OAuth2	Authentification, JWT, rôles
Synchrone	OpenFeign, Resilience4j	Appels REST, circuit breaker
Asynchrone	RabbitMQ, Spring Cloud Stream	Événements métier
Persistance	Spring Data JPA, Spring Data MongoDB	H2, MySQL, MongoDB
Documentation	OpenAPI, Swagger UI	Contrats d'API
Front-end	React	Interface web
Déploiement	Docker, Docker Compose	Lancement de la plateforme

Table 3.3 – Technologies retenues

3.4 Contrats des API REST

3.4.1 Conventions communes

— **URI** : préfixe `/api/v1`, noms au pluriel, filtres dans la chaîne de requête (`?status=`, `?q=`).

— **Verbes HTTP** : **GET** lire, **POST** créer ou déclencher une action, **PUT** remplacer, **PATCH** modifier en partie, **DELETE** supprimer.

— **Codes de réponse** : `200` OK, `201` Created, `204` No Content, `400` données invalides, `401` non connecté, `403` interdit, `404` introuvable, `409` conflit métier, `503` service indisponible.

— **Documentation et erreurs** : OpenAPI et Swagger UI pour chaque service ; erreurs au format *Problem Details* ; routes `/internal` non exposées par la Gateway.

3.4.2 MS1 · Pharmacies (port 8081, H2)

Verbe	Endpoint	Action · accès	Codes
GET	`/pharmacies?lat=&lng=`	Pharmacies proches · public	200
GET	`/pharmacies?onDuty=true`	Pharmacies de garde · public	200
GET	`/pharmacies/{id}`	Détail d'une pharmacie · public	200, 404
POST	`/pharmacies`	Créer la fiche · PHARMACIST	201, 400, 403
PATCH	`/pharmacies/{id}`	Horaires, garde · propriétaire	200, 403, 404
PATCH	`/pharmacies/{id}/status`	Valider / désactiver · ADMIN	200, 403, 404

Événements : aucun. Service de référence appelé par MS3 et MS4 (OpenFeign).

3.4.3 MS2 · Médicaments (port 8082, H2)

Verbe	Endpoint	Action · accès	Codes
GET	`/medicines?q=&categoryId=`	Rechercher · public	200
GET	`/medicines/{id}`	Fiche médicament · public	200, 404
GET	`/medicines/{id}/alternatives`	Génériques · public	200, 404
POST	`/medicines`	Ajouter · ADMIN	201, 400, 409
PUT	`/medicines/{id}`	Modifier · ADMIN	200, 404
DELETE	`/medicines/{id}`	Supprimer · ADMIN	204, 404
GET	`/categories`	Lister les catégories · public	200
POST	`/categories`	Créer une catégorie · ADMIN	201, 409

Événements : aucun. Service de référence appelé par MS3 et MS4 (OpenFeign).

3.4.4 MS3 · Stocks & Disponibilité (port 8083, H2)

Verbe	Endpoint	Action · accès	Codes
GET	`/stocks?medicineId=`	Disponibilité et prix · public	200
GET	`/stocks?pharmacyId=`	Stock d'une pharmacie · PHARMACIST	200
POST	`/stocks`	Ajouter un produit · PHARMACIST	201, 403, 409
PATCH	`/stocks/{id}`	Modifier quantité, prix · PHARMACIST	200, 403, 404
DELETE	`/stocks/{id}`	Supprimer un produit · PHARMACIST	204, 403
POST	`/stocks/{id}/movements`	Réapprovisionner · PHARMACIST	201, 403, 404
GET	`/stocks/{id}/movements`	Historique · PHARMACIST	200
POST	`/internal/v1/stocks/ reservations`	Réserver des quantités · interne (MS4)	201, 409

Événements : publie `stock.replenished` ; reçoit `stock.released`.

3.4.5 MS4 · Commandes & Retraits (port 8084, MySQL)

Verbe	Endpoint	Action · accès	Codes
POST	`/orders`	Passer une commande · PATIENT	201, 409, 503
GET	`/orders?status=`	Suivre ses commandes / commandes reçues · PATIENT, PHARMACIST	200
GET	`/orders/{id}`	Détail · propriétaire	200, 403, 404
PUT	`/orders/{id}`	Modifier (si en attente) · PATIENT	200, 409
DELETE	`/orders/{id}`	Supprimer (si en attente) · PATIENT	204, 409
PATCH	`/orders/{id}/status`	Accepter, refuser, prête, retirée · PHARMACIST	200, 403, 409
POST	`/alerts`	Créer une alerte de stock · PATIENT	201, 409
GET	`/alerts`	Mes alertes · PATIENT, PHARMACIST	200
DELETE	`/alerts/{id}`	Supprimer une alerte · PATIENT	204, 404

Événements : publie `order.events`, `alert.triggered`, `stock.released` ; reçoit `stock.replenished`.

3.4.6 MS5 · Notifications & Messagerie (port 8085, MongoDB)

Verbe	Endpoint	Action · accès	Codes
GET	`/notifications?read=false`	Mes notifications · authentifié	200
PATCH	`/notifications/{id}`	Marquer comme lue · destinataire	200, 403, 404
GET	`/conversations`	Mes conversations · PATIENT, PHARMACIST	200
POST	`/conversations`	Ouvrir une conversation · PATIENT	201
GET	`/conversations/{id}/messages`	Lire les messages · participant	200, 403
POST	`/conversations/{id}/messages`	Envoyer un message · participant	201, 403
GET	`/dashboard/statistics`	Statistiques · ADMIN	200, 403
GET	`/dashboard/events?type=`	Journal des événements · ADMIN	200, 403

Événements : reçoit `stock.replenished`, `alert.triggered`, `order.events` ; aucun appel sortant.

3.5 Communication synchrone et asynchrone

3.5.1 Communication synchrone (OpenFeign)

Le synchrone est choisi lorsque **l'appelant a besoin de la réponse pour continuer**. Les appels passent par OpenFeign, avec résolution du nom par Eureka et un circuit breaker Resilience4j.

Appel		Pourquoi	Si le service appelé tombe
MS3 MS1	→	Afficher le détail de la pharmacie	Mode dégradé, sans le détail
MS3 MS2	→	Vérifier le médicament avant l'ajout au stock	`503`, ajout refusé
MS4 MS2	→	Nom et prix du médicament	`503`, commande refusée
MS4 MS1	→	Vérifier que la pharmacie est active	Mode dégradé
MS4 MS3	→	Réserver les quantités	`409` si stock insuffisant, `503` si panne : rien n'est enregistré

Table 3.4 – Appels synchrones entre microservices

3.5.2 Communication asynchrone (RabbitMQ)

L'asynchrone est choisi pour **diffuser un fait sans bloquer l'émetteur**. L'émetteur ne connaît pas ses consommateurs : les services restent découplés.

Événement	Émetteur	Reçu par	Déclencheur
`stock.replenished`	MS3	MS4, MS5	Réapprovisionnement : déclenche les alertes et les notifications
`order.events`	MS4	MS5	Commande créée, acceptée, refusée, prête, retirée
`alert.triggered`	MS4	MS5	Alerte déclenchée par un retour en stock
`stock.released`	MS4	MS3	Commande refusée ou annulée : les quantités sont rendues

Table 3.5 – Événements métier

3.5.3 Vue des interactions par microservice

Les figures suivantes montrent, pour chaque microservice, sa place dans l'architecture : les appels qu'il reçoit et émet, sa base et les événements qu'il échange.

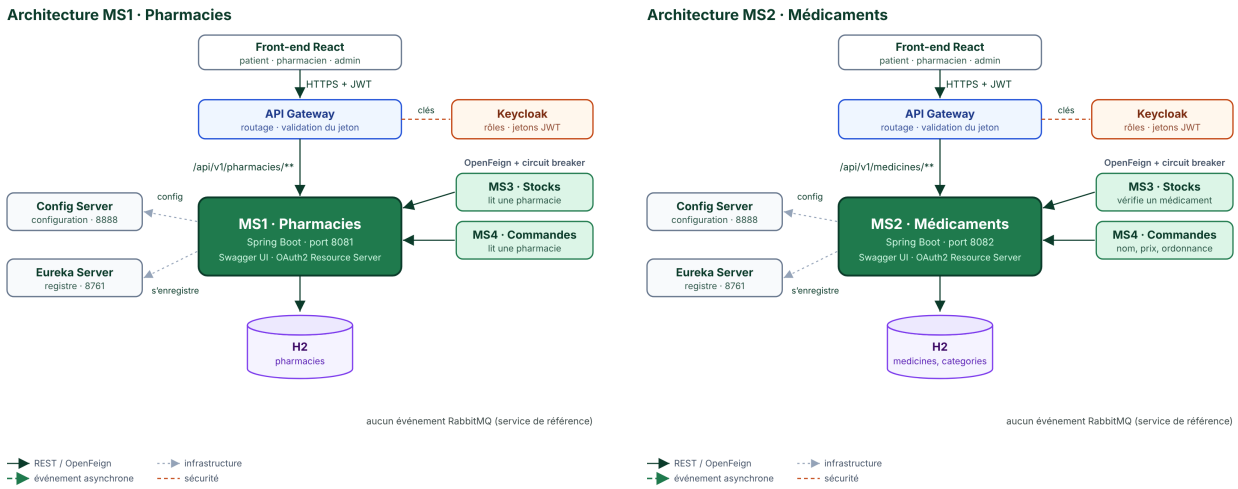


Figure 3.2 – Interactions de MS1 (gauche) et MS2 (droite)

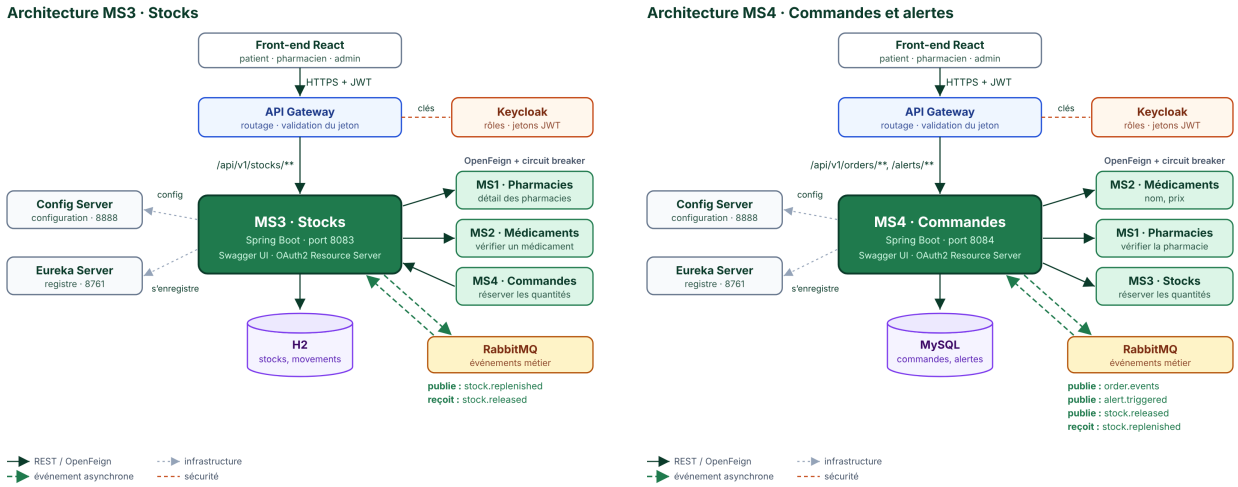


Figure 3.3 – Interactions de MS3 (gauche) et MS4 (droite)

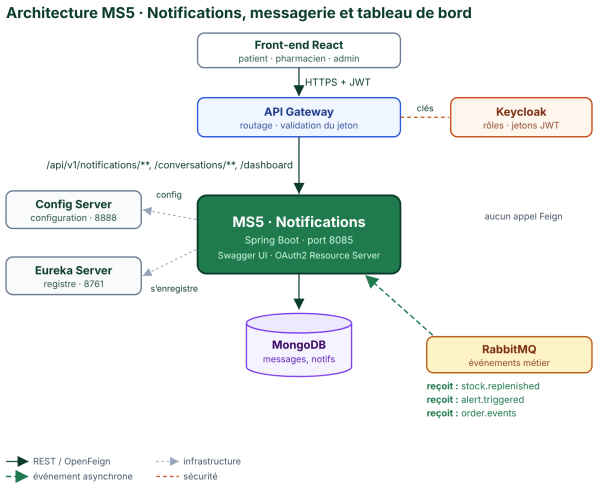


Figure 3.4 – Interactions de MS5

3.6 Sécurité et résilience

3.6.1 Sécurité

— **Keycloak** gère les comptes et les rôles `PATIENT`, `PHARMACIST` et `ADMIN` dans un royaume dédié, exporté dans le dépôt pour être réimporté au démarrage.

— Le front-end s'authentifie par OpenID Connect et envoie le jeton dans l'en-tête `Authorization: Bearer`.

— La **Gateway** vérifie la signature et l'expiration du jeton ; les lectures publiques (recherche) restent accessibles sans compte.

— Chaque **microservice** vérifie à nouveau le jeton, le rôle et le propriétaire de la ressource (un pharmacien ne gère que sa pharmacie et son stock).

— Les routes internes (`/internal/...`) ne sont pas exposées par la Gateway.

3.6.2 Résilience et cohérence

Situation	Mécanisme
Un service appelé ne répond pas	Circuit breaker Resilience4j, timeout et méthode de repli (mode dégradé)
Deux patients veulent la dernière boîte	Décrément atomique : `UPDATE stock SET quantity = quantity - :q WHERE id = :id AND quantity >= :q` ; zéro ligne modifiée signifie `409`
Commande refusée ou annulée	Compensation : `stock.released` rend les quantités à MS3
MS5 arrêté	Files RabbitMQ durables : les messages attendent et sont traités au redémarrage
Message reçu deux fois	Chaque événement porte un identifiant ; un consommateur ignore un message déjà traité

Table 3.6 – Mécanismes de résilience et de cohérence

CHAPITRE 4

Organisation et perspectives

4.1 Répartition des tâches

Membre	Microservice individuel	Partie commune
Jihed Khlifi	MS1 · Pharmacies	API Gateway, Swagger
Abdelhamid Ayadi	MS2 · Médicaments	Keycloak et sécurité
Ferdaouas Oueslati	MS3 · Stocks & Disponibilité	Docker, RabbitMQ, pilotage de MS5
Insaf Makhloufi	MS4 · Commandes & Retraits	Eureka, Config Server

Table 4.1 – Répartition des tâches

Le microservice commun MS5 est développé par toute l'équipe ; chaque membre réalise aussi les pages du front-end liées à son microservice.

4.2 Organisation du dépôt Git et gestion de projet

Dépôt public : `https://github.com/insafmakhloufi/projweb-medfinder-4DASI11`. L'enseignante y est ajoutée comme collaboratrice.

```
projweb-medfinder-4DASI11/
|-- Backend/
|   |-- config-server/          configuration centralisee
|   |-- eureka-server/          registre des services
|   |-- api-gateway/            point d'entree unique
|   |-- pharmacy-service/       MS1 Pharmacies
|   |-- medicine-service/       MS2 Medicaments
|   |-- stock-service/          MS3 Stocks et disponibilite
|   |-- reservation-service/    MS4 Commandes et retraits
|   |-- notification-service/   MS5 Notifications et messagerie
|-- Frontend/                   application React
|-- Documentation/
|   |-- Diagrams/               global + un dossier par microservice
|   |-- Presentation/           support de la validation
|-- README.md                   sujet, membres, architecture, lancement
```

— Branche `main` protégée ; une branche par fonctionnalité ; fusion par *pull request* relue par un autre membre.

— Chaque membre committe lui-même son microservice : l'historique reflète le travail de chacun.

— Suivi des tâches dans un tableau GitHub Projects (À faire, En cours, En revue, Terminé).

4.3 Planification du projet

Semaine	Objectif
S4	Validation de la conception et projets créés dans le dépôt
S5	Entités, CRUD, Gateway, Eureka, Config Server
S6	OpenFeign et circuit breaker
S7	RabbitMQ, événements et MS5
S8	Sécurité Keycloak
S9	Docker et front-end React
S10	Tests et démonstration finale

Table 4.2 – Planning prévisionnel

4.4 Risques et mesures

Risque	Mesure
Configuration Keycloak complexe	Royaume exporté dans le dépôt et importé au démarrage
Survente de la dernière boîte	Décrément atomique et compensation
Service indisponible	Circuit breaker, mode dégradé, files durables
Retard d'un membre	Backlog partagé, revues croisées, réaffectation des tâches

Table 4.3 – Risques identifiés et mesures prévues

Conclusion et prochaines étapes

MedFinder facilite la recherche et la localisation des médicaments disponibles dans les pharmacies participantes, tout en assurant une gestion efficace des stocks et des commandes grâce à une architecture sécurisée et évolutive basée sur les microservices.

La conception présentée dans ce rapport repose sur cinq microservices issus des contextes métier, chacun propriétaire de sa base (H2, MySQL, MongoDB), sur une infrastructure Spring Cloud complète (Gateway, Eureka, Config Server), sur une sécurité centralisée avec Keycloak et sur une communication mixte : synchrone avec OpenFeign lorsque la réponse est nécessaire, asynchrone avec RabbitMQ pour diffuser les faits métier. Les comportements en cas de panne et les mécanismes de cohérence sont définis dès cette étape.

Prochaines étapes : implémenter les entités et les CRUD dès la semaine 5, puis la communication entre services, la messagerie RabbitMQ et le microservice commun, la sécurité Keycloak, la conteneurisation Docker et le front-end, jusqu'à la démonstration finale en semaine 10.

ANNEXE A

Exemples d'échanges

A.1 Passer une commande

Requête `POST /api/v1/orders` (jeton du patient) :

```
{ "pharmacyId": 1, "lines": [ { "medicineId": 9, "quantity": 2 } ] }
```

Réponse `201 Created`, en-tête `Location: /api/v1/orders/42` :

```
{
  "id": 42, "patientUsername": "patient1", "pharmacyId": 1,
  "status": "PENDING", "pickupCode": "MF-7K3Q9", "totalAmount": 9.60,
  "lines": [ { "medicineId": 9, "medicineName": "Doliprane 1000 mg",
               "quantity": 2, "unitPrice": 4.80 } ],
  "createdAt": "2026-11-25T10:15:00Z"
}
```

A.2 Erreur : stock insuffisant

```
{
  "type": "https://medfinder.local/errors/insufficient-stock",
  "title": "Stock insuffisant",
  "status": 409,
  "detail": "Only 1 unit left for medicine 9 in pharmacy 1",
  "instance": "/api/v1/orders"
}
```

A.3 Événement `stock.replenished`

```
{
  "eventId": "6f1c2e0a-...", "type": "stock.replenished",
  "stockId": 17, "pharmacyId": 1, "medicineId": 9,
  "medicineName": "Doliprane 1000 mg", "quantity": 30,
  "occurredAt": "2026-11-25T08:00:00Z"
}
```

A.4 Événement `order.events`

```
{
  "eventId": "a93d...", "type": "ORDER_ACCEPTED",
  "orderId": 42, "patientUsername": "patient1", "pharmacyId": 1,
  "status": "ACCEPTED", "occurredAt": "2026-11-25T10:20:00Z"
}
```